

Learning-based Path Planning and Tracking for the Walking–Jumping Quadrupedal Robot[★]

Zhehan Tao^a, Zimeng Li^a, Erwin Ang Tien Yew^a, Shuaishuai Sun^{a,*} and Liang Zhang^{b,**}

^aUniversity of Science and Technology of China, Hefei, China

^bAnhui University, Hefei, China

ARTICLE INFO

Keywords:

Quadruped robot

Path planning

Path tracking

Reinforcement learning

ABSTRACT

Walking–jumping hybrid quadruped robots enable traversal over discontinuous terrain, but existing navigation pipelines often suffer from a mismatch between high-level planning and low-level execution, especially for highly dynamic maneuvers such as jumps. We propose a capability-aware multimodal planning and tracking framework that aligns global path generation with reinforcement-learning-based locomotion execution for walking–jumping quadrupeds. Our planner augments elevation-map RRT planning with discrete optimization and smoothing, and refines jump takeoff and heading decisions using feasibility constraints derived from the robot's empirically validated jump-tracking range, producing executable multimodal paths. A tracking module streams velocity and landing-target commands and uses a dual-condition trigger based on gate crossing and takeoff distance to achieve reliable mode switching and jump initiation under localization uncertainty. Vision-free walking and jumping controllers are trained via two-stage curriculum learning with Perlin-noise terrain randomization and deployed with a lightweight onboard inference stack. Experiments in simulation and on a Unitree Go1 equipped with a single LiDAR demonstrate reliable long-range navigation, achieving a 98.61% within-corridor tracking rate in complex environments and 93.38% on discontinuous terrain with mixed walking and jumping under resource-limited onboard compute.

1. Introduction

Walking–jumping hybrid quadruped robots extend the locomotion capability of conventional legged systems and enable traversal over discontinuous terrain. However, effectively leveraging these multimodal capabilities within a unified navigation pipeline remains challenging. A key limitation is the mismatch between high-level planning and low-level execution: global planners often optimize geometric or terrain objectives without explicitly accounting for the robot's reliable capability region for dynamic maneuvers, while learning-based locomotion controllers typically track generic commands and provide limited mechanisms to guarantee faithful execution of complex multimodal paths. This gap reduces path executability and degrades tracking reliability in real-world deployments, especially when sensing and onboard compute are limited.

For global planning that combines walking and jumping, Norby et al. [1] proposed a dynamics-based RRT-Connect [2] planner (KD-RRT-Connect). It treats a “walk–jump” sequence as an action primitive and expands the search tree

by executing one primitive, enabling multimodal planning with both walking and jumping.

Wellhausen et al. [3] used a probabilistic roadmap that considers terrain traversability and footholds in complex terrain, which can fully leverage the robot's walking capability. However, many traditional planners generate feasible paths without explicitly reasoning about the robot's jumping performance. As a result, they may avoid obstacles that could be crossed or gaps that could be jumped over, producing suboptimal routes.

Xu et al. [4] considered the robot's jumping ability and proposed a planning method for environments with height variations, but did not provide a corresponding tracking framework. Moreover, their jump feasibility check does not incorporate performance factors such as landing-target tracking performance and takeoff posture constraints. Yue et al. [5] utilized online RRT planning and detected abrupt height changes along the path, switching to a jump-trajectory optimization module upon detection. However, the method does not plan a complete global trajectory; instead, it adjusts the path based on landing points, which reduces practicality and tracking reliability.

Recently, reinforcement-learning-based navigation has largely adopted hierarchical frameworks [6]. Hoeller et al. [7] reconstruct the surrounding terrain using multiple sensors, with fine reconstruction in reachable areas and coarse reconstruction in distant regions. The robot then selects among pre-trained skills such as crouching, jumping, and climbing to complete planning and tracking on discontinuous terrain.

Kim et al. [8] proposed a high-speed planning and tracking framework for discrete terrain that determines the route by filtering footholds through a three-stage screening

[★]Funding: This work was supported in part by the National Natural Science Foundation of China under grant No. 62303002, the State Key Laboratory of Micro-Spacecraft Rapid Design and Intelligent Cluster under grant No. MS01240110, the Key Program of Natural Science Foundation of Anhui Higher Education Institutions of China under grant No. 2022AH050068, the Major Project of Anhui Province's Science and Technology Innovation Breakthrough Plan (No. 202423h08050003), and the Fundamental Research Funds for the Central Universities (No. YD20900002019).

*Corresponding author

**Corresponding author

✉ tzh992171976@mail.ustc.edu.cn (Z. Tao); sssun@ustc.edu.cn (S. Sun); liangzhang@ahu.edu.cn (L. Zhang)

ORCID(s):

pipeline. In simulation, they co-train the planning and motion-control policies with the map, achieving faithful tracking of planned footholds at high speed. However, learning-based planning and tracking are often strongly coupled to the simulated training environment, which can lead to poor transfer to real-world unstructured terrain. Moreover, these methods may require rich sensing and significant onboard compute, and the resulting routes can be less interpretable and controllable for operators. Finally, because hierarchical pipelines involve policy invocation and distillation, modifying any stage can require retraining the full stack, incurring substantial time cost.

To address these issues, we combine offline global planning, which is long-range and controllable, with reinforcement-learning-based locomotion control and a lightweight onboard deployment stack, organized as a deployment-driven framework in which each layer is designed around the constraints of the others. We propose a capability-aware framework for multimodal planning and tracking on walking-jumping quadrupeds: the planner augments sampling-based planning with discrete optimization and smoothing, and refines jump-related decisions using feasibility constraints derived from the robot's validated tracking performance; the tracker streams velocity and landing-target commands and uses a dual-condition trigger to initiate jumps at the planned takeoff pose; the locomotion module executes commands with vision-free walking and jumping policies trained via two-stage curriculum learning with Perlin-noise terrain randomization and deployed with a lightweight onboard inference stack based on MNN (Mobile Neural Network) [29]. We validate the approach on a Unitree Go1 robot in simulation and real-world experiments, achieving a 98.61% long-distance within-corridor tracking rate in complex environments and 93.38% on unstructured discontinuous terrain with mixed walking and jumping. The main contributions are summarized as follows:

- We propose an optimized walking-jumping global path planning method. Instead of triggering jumps passively wherever the elevation map exhibits abrupt height changes, the planner jointly optimizes path length, turning variation, center-of-mass elevation change and avoids unnecessary jumps, and refines the takeoff pose, heading, and landing target of each retained jump within the executing policy's empirically validated jump-feasibility region, which further reduces the tracking error. A dual-condition trigger initiates each jump at the planned takeoff pose, preserving planning-tracking consistency under LiDAR-odometry uncertainty.
- For single-LiDAR perception, we design tailored reward functions trained through two-stage curriculum reinforcement learning: the walking policy is trained with body-balance and abrupt-acceleration penalties that suppress the attitude fluctuations corrupting LiDAR odometry, while the jumping policy is trained

with takeoff-posture constraints and landing-target rewards for stable, obstacle-safe landing. In addition, a variable-weight balance method scales the stability-oriented rewards according to the convergence of the tracking rewards, so that the agility-related and posture-balance rewards are optimized synchronously instead of competing with each other. These designs reduce the instability caused by single-sensor exteroception and reduce the tracking error.

- We integrate the complete system on resource-limited hardware and validate it experimentally. Under single-LiDAR perception, the walking and jumping policies run as finite-state machines through a lightweight MNN-based inference stack on a single Jetson Nano, and the system is demonstrated in long-range real-world navigation over complex and discontinuous terrain. To the best of our knowledge, this is the lowest-cost integrated navigation-and-tracking solution for walking-jumping quadrupeds reported to date.

2. Related Works

2.1. Global Path Planning For Quadrupedal Robots

Common quadruped path-planning methods can be broadly divided into two categories. The first category uses global planning to generate a path, followed by a motion planner to produce foot trajectories and footholds. Global planners are mainly categorized into search-based and sampling-based methods [9]. Sampling-based planners, such as Rapidly-exploring Random Trees (RRT) [10] and Probabilistic Roadmap (PRM) [11], often offer lower computational cost for high-DoF planning in complex environments. Compared with 2D grid maps, elevation maps encode height and slope information and provide richer terrain feedback [12]. Therefore, combining sampling-based planning with elevation mapping is a common solution for quadruped navigation.

The second category is foundation-model-based navigation, which is also typically hierarchical. Multiple locomotion skills are first pre-trained to handle diverse terrain; a perception or reasoning module then segments the route into sub-waypoints, and the system composes skills to reach each waypoint and complete navigation. Shah et al. [15] combine pre-trained language, vision, and action models to navigate toward natural-language goals, and Yokoyama et al. [16] build vision-language frontier maps for zero-shot semantic navigation. For legged robots, Wang et al. [13] and Zhou et al. [14] employ large language models for autonomous navigation through interactive, hierarchical control, and Cheng et al. [17] couple a vision-language-action model with real-time locomotion policies. Despite their flexibility, such methods face practical limitations for onboard legged deployment: large models may suffer from hallucinated or weakly grounded waypoint decisions; their inference latency and memory footprint strain resource-limited onboard computers; and they may

not explicitly encode robot-specific feasibility and safety constraints, which complicates repeatable execution on discontinuous terrain. In contrast, our framework performs reasoning offline at the global-planning level and keeps onboard execution lightweight and deterministic.

2.2. Learning for Legged Locomotion

In recent years, quadruped locomotion control has been increasingly dominated by reinforcement learning. Building upon Rapid Motor Adaptation (RMA) [18], Dreamwaq [19] trains a foundational locomotion controller that estimates external terrain characteristics from proprioception, enabling robust walking without exteroceptive sensing. Furthermore, Luo et al. [20] introduced PIE, which integrates robot vision on top of the estimated environment features and improves robustness on highly discrete terrain with limited footholds.

Regarding jump training for legged robots, many mainstream approaches rely on reference-trajectory generation, such as the methods by Zhang et al. [21] and Bussola et al. [22]. These methods train the robot to track the generated trajectories toward different landing targets. In contrast, Atanassov et al. [23] adopted a purely reward-driven formulation and used curriculum learning to train a jumping policy that adapts to landing conditions and can handle obstacles. More recently, parkour-style controllers have pushed the agility boundary of small quadrupeds: Zhuang et al. [24] compose multiple parkour skills to traverse challenging obstacle courses, and Cheng et al. [25] train a single end-to-end policy from depth images that leaps across gaps up to twice the body length. These methods demonstrate impressive agility, but the learned skills are largely reactive and are not connected to global path planning or landing-tracking.

3. Method

As shown in Fig. 1, our system comprises three layers: global path generation, planning-consistent tracking, and learning-based locomotion execution. The planner first produces a collision-free multimodal route on a terrain elevation map using RRT-style sampling, and then improves executability via discrete sampling optimization, curve fitting, and smoothing. For each jump segment, the planner further refines the takeoff pose and heading using feasibility constraints derived from the robot's validated tracking range. The navigation layer consists of a command publisher and a lightweight tracking state machine. The publisher streams velocity commands along the planned path and, when approaching a planned takeoff pose, publishes a jump command together with the landing target in the takeoff frame. The tracking state machine selects the appropriate locomotion mode (walking or jumping) and forwards the corresponding command interface to the controller. The locomotion layer contains two reinforcement-learning policies (walking and jumping) deployed as finite-state machines. Policy outputs are filtered and converted into target joint position references that are tracked by a low-level PD controller on the robot.

3.1. Global Path Generation

Our global planner takes as input an elevation map M and a start/goal pair (p_s, p_g) . In simulation, M is directly available from the environment. In real-world experiments, M is built online from the onboard LiDAR point cloud in Fig. 1 and updated at a fixed rate; obstacle boundaries and height discontinuities are extracted from M by thresholding local height gradients and segmenting connected components. The output is a sequence of waypoints with associated locomotion mode labels and, for each jump, a takeoff pose and a landing target.

The path generation consists of four steps, **Step 1 (pre-processing and jump-point refinement)**. Input: a coarse multimodal path from KD-RRT(-Connect) on the elevation map. Output: a cleaned path with refined jump segments. Due to the randomness of KD-RRT, unnecessary jumps may be planned. We add a height-variation constraint during jump detection: if the height difference between two consecutive sampled nodes is less than a threshold, that segment is replanned as walking; otherwise it is retained as a jump. We then refine each planned jump by adjusting both takeoff distance to the obstacle boundary and takeoff direction based on the robot's empirically validated jump-tracking range. In particular, for "jumping onto" an obstacle, the takeoff point is moved away from the obstacle boundary by a safe distance:

$$\begin{bmatrix} x_{\text{jump}} \\ y_{\text{jump}} \end{bmatrix} = \begin{bmatrix} x_{\text{boundary}} \\ y_{\text{boundary}} \end{bmatrix} - \text{safe_distance} \times \begin{bmatrix} \cos \theta_{\text{boundary}} \\ \sin \theta_{\text{boundary}} \end{bmatrix} \quad (1)$$

Here $(x_{\text{boundary}}, y_{\text{boundary}})$ and θ_{boundary} denote the closest boundary point and its tangent direction extracted from the obstacle boundary on M , and safe_distance is chosen based on the robot's body length and takeoff clearance margin. Subsequently, we adjust the takeoff orientation to ensure the takeoff pose and jump direction remain within the robot's validated jump-tracking range. We add a hard constraint during RRT expansion, making the angle difference $\Delta\theta$ smaller than the maximum allowable angle θ_{max} , and incorporate a soft constraint in the following optimization steps. The penalty term is:

$$C_{\text{jump}} = w_j \cdot \max(0, |\Delta\theta| - \theta_{\text{max}})^2 \quad (2)$$

Step 2 (discrete space sampling). Input: the preprocessed path from Step 1 and the elevation map M . Output: a coarse but locally optimized discrete path. We perform intensive discrete sampling on both sides of the preprocessed path to generate candidate point sets, and evaluate them using a terrain cost that considers distance, height difference, turning angle, slope change, goal proximity, and the jump-direction penalty. The transition cost is:

$$\begin{aligned} \text{Cost}_{\text{trans}} = & \text{Cost}_{\text{dis}} + \text{Cost}_{\text{dz}} + \text{Cost}_{\text{ang}} \\ & + \text{Cost}_{\text{slope}} + \text{Cost}_{\text{goal}} + \text{Cost}_{\text{jump}} \end{aligned} \quad (3)$$

where each cost term is the product of a defined weight and the corresponding sampled value. Next, we use a dynamic

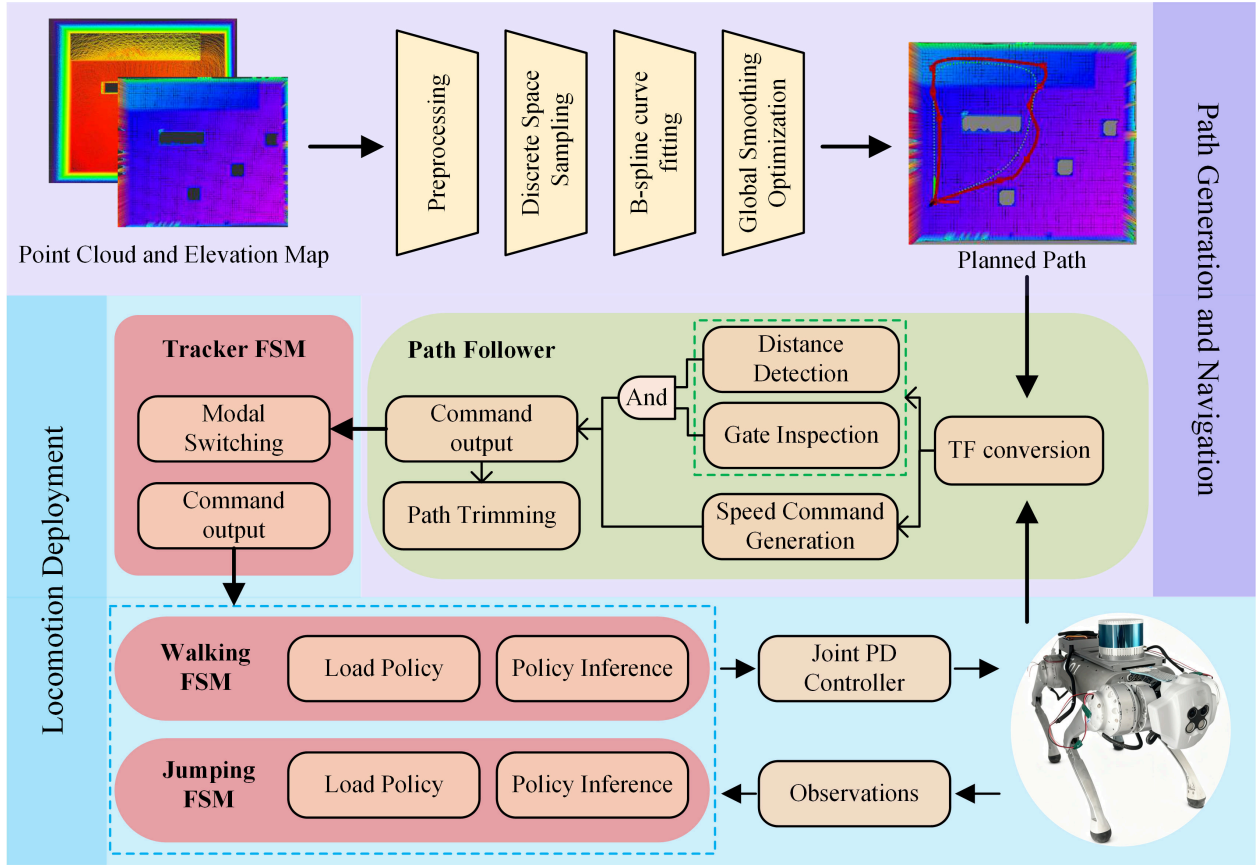


Figure 1: Framework overview. The path-generation stack converts LiDAR point clouds into an elevation map, then applies preprocessing, discrete sampling, B-spline fitting, and quadratic-programming smoothing to obtain a smooth, feasible multimodal path. The command publisher streams velocity commands along the path; when the dual-condition trigger based on gate crossing and takeoff distance confirms the takeoff pose, it issues a jump command. The tracking state machine routes commands to the walking or jumping controller. Each policy outputs target joint positions, which are low-pass filtered before actuation. Proprioceptive state and pose estimates are fed back to motion control and the publisher.

programming algorithm to search for the coarse solution path with the minimum total cost among all feasible sampling points. This enables the algorithm to actively select flatter routes with gentler turns while ensuring reachability.

Step 3 (B-spline curve fitting). Input: the coarse discrete path from Step 2. Output: a smooth reference curve. We fit a cubic B-spline to transform discrete points into a smooth continuous curve, leveraging its local control and continuity. A k -th order B-spline curve is:

$$P(u) = \sum_{i=0}^n P_i B_{i,k}(u) \quad (4)$$

In this formulation, P_i denotes the control points initialized from the discrete path, and $B_{i,k}(u)$ is the basis function defined recursively following the de Boor–Cox definition [26]. To obtain a uniformly spaced reference suitable for control, we resample the fitted spline at fixed arc-length intervals to produce a sequence of smooth, uniformly distributed path points. **Step 4 (QP-based global smoothing).** Input: the reference curve from Step 3 and a collision-free corridor inferred from M around the reference curve. Output: the final smoothed path. We formulate smoothing as a quadratic

program:

$$\begin{cases} \min \left(\frac{1}{2} \mathbf{x}^T \mathbf{H} \mathbf{x} + \mathbf{g}^T \mathbf{x} \right) \\ \text{s.t. } \mathbf{lb} \leq \mathbf{A} \mathbf{x} \leq \mathbf{ub} \end{cases} \quad (5)$$

where $\mathbf{x}$ is the optimization variable, $\mathbf{H}$ is the Hessian matrix, $\mathbf{g}$ is the gradient vector, $\mathbf{A}$ is the constraint matrix including equality and inequality constraints, and $\mathbf{lb}$ and $\mathbf{ub}$ are the lower and upper bounds, respectively. The objective balances path smoothness, compactness, and fidelity to the reference curve:

$$Cost = Cost_{smooth} + Cost_{length} + Cost_{dev} \quad (6)$$

Additionally, we optimize the height of the path points independently to reduce frequent vertical centroid adjustments. Finally, we use OSQP to solve the QP and obtain a final path optimized in both horizontal and vertical directions. After optimization, we re-check the path against the elevation map corridor to ensure it remains collision-free; if any violation is detected, we reduce the corridor width and re-solve.

3.2. Path Tracking and Locomotion Module

Switching

Path tracking comprises a command publisher and a tracking state machine. The command publisher subscribes to the discrete path points produced by the planner in the elevation-map frame, estimates the robot pose using LiDAR odometry and the known initial pose, and computes linear velocity commands in x , y , and z and angular velocity commands based on the nearest path point and a nominal forward speed. To maintain faithful tracking and avoid large transients during mode switching, we cap the tracking speed at 0.8 m/s and use 0.4 m/s as the nominal speed.

Since the planned path contains no intermediate points along a jump segment, whereas points along walking segments are spaced at 0.1 m, we identify a jump takeoff point when the distance between two consecutive points exceeds 0.3 m. When the publisher detects that the next point is a takeoff point, it stops publishing planar commands in x , y , and yaw and instead publishes a fixed z -channel flag that indicates a jump command and the planned landing target relative to the takeoff point. If the jump command is issued at a mismatched position, severe tracking errors can occur. To initiate the jump at the planned takeoff pose, we employ a dual-condition trigger with two complementary conditions. The first condition is geometry-based gate detection: a virtual gate is defined at the takeoff point, and the jump is triggered when the robot crosses the gate plane in the correct direction and the lateral offset is within tolerance. This direction-sensitive test prevents unintended triggering under incorrect motion states. The second condition is distance-based detection: the jump is triggered when the robot-to-takeoff distance enters a threshold band; by tracking the historical minimum distance, this condition also detects missed events and provides a compensatory trigger. The publisher issues a jump command only when both conditions are satisfied.

The tracking state machine receives commands from the command publisher, with the walking state machine as the default. We implement a minimal finite-state machine with four states: *WalkTrack*, which follows planar velocity commands; *PreJumpStand*, which switches to a fixed standing posture and stabilizes base attitude; *JumpExecute*, which runs the jump policy with a landing-target input; and *Recovery*, which re-acquires stable contact and returns to walking. The transition *WalkTrack*→*PreJumpStand* occurs only when the dual-condition trigger fires at the planned takeoff pose. The transition *PreJumpStand*→*JumpExecute* occurs after a fixed stabilization duration and within an attitude tolerance. The transition *JumpExecute*→*Recovery* occurs upon touch-down detection. Finally, *Recovery*→*WalkTrack* occurs when the base height and velocity tracking return within thresholds. The command publisher continues streaming velocity commands while the robot is executing a jump.

3.3. Locomotion Training

The locomotion controller consists of two parts: walking and jumping. Each is implemented as a finite-state machine using an MNN-based onboard inference framework, with policies trained via reinforcement learning to track the commands produced by the command publisher. Both are “blind” policies that do not use visual inputs. This design is appropriate because the planner determines the route and the locomotion modes; incorporating vision could encourage the policy to make independent navigation decisions and deviate from the planned route. Because our exteroception relies on a single LiDAR, odometry error is reduced by stabilizing the body against abrupt pitch/roll changes and self-occlusion while maintaining fast and faithful velocity-command tracking. We incorporate corresponding reward terms during training to promote these behaviors.

Command Inputs (o_c)	Locomotion		Jumping	
	Base velocities	R^3	Obstacle information	R^6
	Base height	R^1	Landing target	R^3
	Foot swing height	R^1	Landing orientation	R^4
	Gait frequency	R^1	Jump signal	R^1
Observations (o_t)	Base angle velocities	R^{3*N}	Base angle velocities	R^{3*N}
	Base line velocities	R^{3*N}	Base line velocities	R^{3*N}
	Joint position	R^{12*N}	Joint position	R^{12*N}
	Joint velocities	R^{12*N}	Joint velocities	R^{12*N}
	Last Actions	R^{12*N}	Feet contact	R^{4*N}
	Body gravity	R^{3*N}	Base orientation	R^{4*N}

Figure 3: Observation Space for Walking and Jump Policies

3.3.1. Walking Policy

We adopt a two-stage training approach (Fig. 2). The first stage trains a robust walking policy. We follow Walk-These-Ways (WTW) [27] and Dreamwaq [19], using Multiplicity of Behavior (Mob) with different gait frequencies and Rapid Motor Adaptation [18] to improve generalization without visual input. The policy can be described as $\pi_t = [O_c^H, O_t^H, z_t, v_t]$, as shown in Fig. 3, where $O_c^H \in \mathbb{R}^6$ and $O_t^H \in \mathbb{R}^{63 \times 20}$ denote the command input and the stacked history over 20 time steps, z_t is the context estimation parameter, and v_t is the estimated base linear velocity. Since the robot is only required to perform walking, we restrict the gait to trot.

Stage 2 trains the robot to maintain balance on challenging terrain while stabilizing its acceleration, which reduces path deviation under abrupt accelerations caused by external disturbances. We therefore introduce a body-balance term, an abrupt-acceleration penalty, and an adaptive speed-limit reward, as listed in Table 1. Specifically, we limit acceleration by upper bounds τ_{lin} and τ_{ang} , and constrain the commanded speed to $[-1, 1]$ m/s to stabilize tracking and emphasize mid-range commands. The deceleration reward R_{brake} encourages the robot to decelerate when it experiences involuntary acceleration due to slipping or tripping.

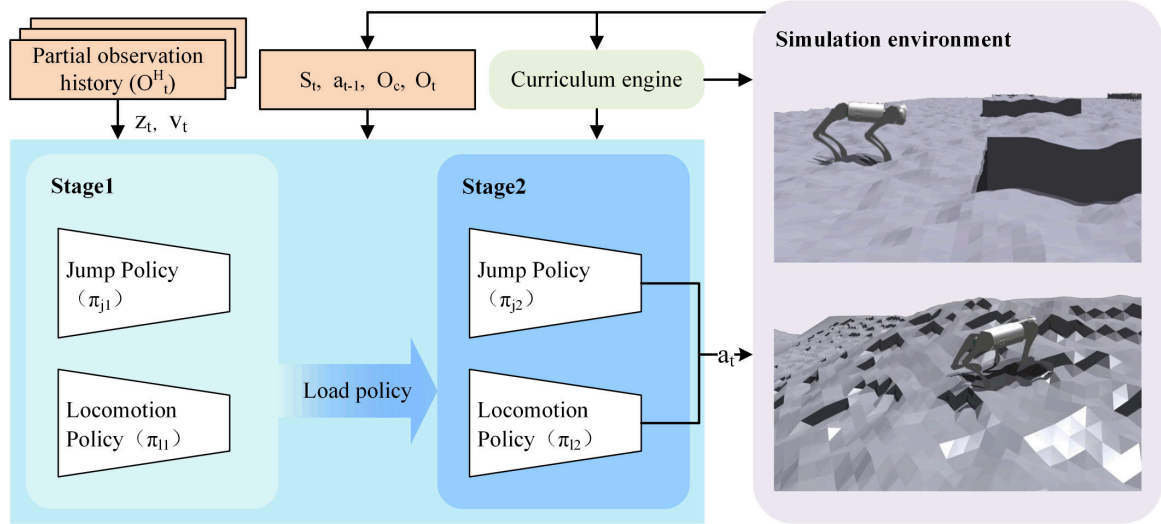


Figure 2: Overview of training stage 2: load the stage-1 pre-trained policy and enable competitive co-evolution between the controller and terrain parameters through the curriculum engine.

In Table 1, ϕ_x and ϕ_y denote the robot base roll and pitch angles, and $v_x, v_y, \omega_x, \omega_y$ denote commanded and tracked planar linear/angular velocities.

To better approximate real-world unstructured terrain and improve body stability, we add sloped terrain and Perlin-noise terrain [28]. Terrain parameters are co-trained with the controller using a curriculum learning scheme. We further add two reward terms to improve motion stability, as listed in Table 1. The body-balance penalty accumulates over time; within one rollout, the penalty increases with both the magnitude and the duration of deviations beyond the balance threshold. To ensure that training prioritizes velocity tracking before emphasizing balance, we define the following adjustment factor for the balance reward:

$$c_{\text{stable}} = \text{clip}(\text{EMA}(R_{\text{tracklin}} + R_{\text{trackang}})/G_{\text{target}}, 0, 1) \quad (7)$$

where G_{target} is the desired maximum convergence value of the reward function. When the tracking reward approaches convergence, the curriculum engine increases terrain difficulty. The domain randomization and curriculum ranges used in training are listed in Table 2.

Table 1
Stage-2 Reward structure

Reward	Expression	Weight
Walking-related terms		
R_{acclimit}	$-\max(0, \ a_{\text{lin}}\ - \tau_{\text{lin}})$	-1.2
R_{brake}	$-0.5 \cdot \max(0, \ a_{\text{ang}}\ - \tau_{\text{ang}})$	-2.0
R_{stable}	$\sum c_{\text{stable}} \cdot \exp(-(\phi_x^2 + \phi_y^2)/\sigma_{\text{stable}})$	-0.5
Jump-related terms		
R_{tracklin}	$\sum \exp(-((v_x - v_{xd})^2 + (v_y - v_{yd})^2)/\sigma_{\text{lin}})$	20
R_{trackang}	$\sum \exp(-((\omega_x - \omega_{xd})^2 + (\omega_y - \omega_{yd})^2)/\sigma_{\text{ang}})$	10
R_{height}	$\mathbb{1}_{\text{hasjumped}} \cdot \sum c_{\text{jump}} \cdot (h_{\text{max}} - 0.5)$	5e3
R_{hasjump}	$\mathbb{1}_{\text{hasjumped}}$	3e2
R_{stablej}	$\sum c_{\text{stablej}} \cdot \exp(-(\phi_x^2 + \phi_y^2)/\sigma_{\text{stablej}})$	1.5
R_{symmetry}	$\sum \exp(-(\sum (q_{\text{left}} - q_{\text{right}})^2)/\sigma_{\text{sm}})$	-3
R_{default}	$\sum \exp(-(\sum (q_j - q_{\text{jdefault}})^2)/\sigma_{\text{dof}})$	15
R_{trackpos}	$\exp(-((x - x_d)^2 + (y - y_d)^2)/\sigma_{\text{pos}})$	3
R_{trackori}	$\exp(-(\phi_z - \phi_{zd})/\sigma_{\text{ori}})$	7
R_{landpos}	$\sum \exp(-((x - x_d)^2 + (y - y_d)^2)/\sigma_{\text{landpos}})$	1.5e3
R_{landori}	$\sum \exp(-(\phi_z - \phi_{zd})/\sigma_{\text{landori}})$	1.2e3
R_{standh}	$\sum \exp(-((h_t - h_{\text{terrain}} - 0.3)^2)/\sigma_{\text{standh}})$	20
R_{feetdis}	$-\sum (p_{\text{foot}} - p_{\text{footdes}})^2 \quad (h > 0.5)$	-20
$R_{\text{frontfeet}}$	$\exp(-((t_{\text{peak}} - t_{\text{front}})^2)/\sigma_{\text{frontfeet}})$	5e6
R_{swing}	$\mathbb{1}_{\text{nearpeak}} \cdot \exp(-((q_t - q_{\text{fdes}})^2)/\sigma_{\text{swing}})$	2e3
R_{energy}	$-\ \tau\ \cdot \omega$	-1e3
R_{change}	$\exp(-(\ \text{contact}_t - \text{contact}_{t-1}\ ^2)/4)$	10
$R_{\text{ct forces}}$	$-\sum \max(0, \ F_{\text{ct}}\ - F_{\text{ct max}})$	-5
$R_{\text{actionrate}}$	$-(a_t - a_{t-1})^2$	-0.2
R_{dofacc}	$-(\dot{q}_{t-1} - \dot{q}_t)/dt)^2$	-1e6
$R_{\text{doflimits}}$	$-\sum (\max(0, q_{\text{min}} - q) + \max(0, q - q_{\text{max}}))$	-10

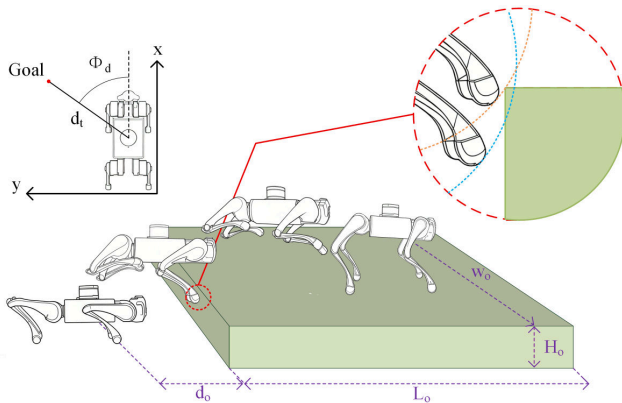


Figure 4: Schematic of Jump Training: The robot's jump command input includes the target distance d_t , direction ϕ_t , and relevant obstacle information d_o, L_o, H_o, W_o . During the obstacle clearance jump, the robot retracts its legs near the apex of the jump to increase the success rate of landing on obstacles.

3.3.2. Jumping Policy

For jumping, we similarly divide policy learning into two stages. The first stage learns the basic jumping motion following the curriculum-learning method of [23]. Since the takeoff behavior learned in the initial stage directly shapes the final takeoff motion, we constrain the takeoff posture from the first stage. This includes setting roll/pitch angle bias penalties and adding termination conditions that check the pre-jump posture – ensuring the robot’s body remains level during takeoff. The second stage trains the robot to perform forward and diagonal jumps with landing-target tracking in the presence of height variations. The command space and observation space are shown in Fig. 3. Since the elevation map and the planned jump/landing points are available beforehand, we simplify the command space by removing the z-axis component and keeping the maximum jump height constant, requiring only landing-point tracking. Experiments show that this simplification also improves landing-target tracking and the obstacle-clearance success rate. We found that synchronous retraction of all four legs during the jump could cause the front legs to contact obstacles during forward jumps, leading to failure. Since reliable landing requires the body to remain as level as possible, collision cannot be avoided by increasing the pitch angle during takeoff. To solve this problem, we add an asynchronous front-leg retraction reward $R_{\text{frontfeet}}$, allowing the robot to retract its legs closer to the highest point during the forward jump, together with a leg-retraction amplitude reward R_{swing} , ensuring the robot’s feet are as close to the body as possible near the highest point. This setup can significantly increase the success rate of jumping onto obstacles. A schematic diagram of the robot’s jumping action is shown in Fig. 4. For the jump height and body balance reward function settings, we also prioritize landing point tracking as the primary task, thus similarly defining:

$$c_{\text{stablej}} = \text{clip}(\text{EMA}(R_{\text{trackpos}} + R_{\text{trackori}})/G_{\text{target}}, 0, 1) \quad (8)$$

Similar to walking-policy training, to improve jumping stability on complex terrain and enable jumps across height variations, we introduce Perlin-noise terrain curricula and obstacle curricula. Unlike Perlin-noise randomization, obstacle curricula are introduced only after the reward for flat-ground landing-target tracking converges, and the difficulty is increased gradually as training progresses.

3.4. Deployment

For long-distance tracking, jitter caused by policy deployment can significantly reduce motion stability, and direct policy output is prone to excessive commands, posing deployment risks. Therefore, we adopt an inference-framework deployment method, which reduces jitter by fixing the inference cycle at 50 Hz and decoupling it from asynchronous middleware threads, and aligning observation timestamps to the action cycle. We apply two practical signal-conditioning steps before sending joint references to the low-level PD loop. First, we use first-order hold interpolation, to upsample the 50 Hz action sequence to the actuator

update rate when needed. Second, we apply a low-pass filter on target joint positions to suppress high-frequency components that are typically caused by estimation noise and network output discontinuities. In this paper, reducing sensor noise refers specifically to mitigating the effect of LiDAR-odometry jitter and time misalignment on policy observations such as velocity, body orientation, and foot contact states by synchronizing message timestamps and filtering the resulting observation streams. In the training setup, we did not randomize K_p and K_d . For hardware deployment, we tune PD gains in a staged procedure: start from the manufacturer-recommended gains, then increase K_p until tracking becomes responsive but before oscillation appears, and finally increase K_d to damp oscillations while keeping sufficient bandwidth for 0.4–0.8 m/s tracking. In our real-robot experiments in Sec. IV, we use $K_p = 20$ and $K_d = 0.5$. To reduce the computational load on the onboard CPU and enable stable onboard execution, we use an MNN-based [29] C-language deployment framework for policy inference. Both walking and jumping policies are deployed as state machines, and the state-machine program is optimized to reduce computational complexity. The final state machine outputs target motor positions at a frequency of 50 Hz. On Jetson Nano, the average end-to-end policy inference latency is on the order of a few milliseconds.

4. Experiments

Simulation Setup: We trained policies in the open-source IsaacGym [30] simulator. For both walking and jumping, we trained 4096 domain-randomized agents in parallel with the randomization and curriculum ranges in Table 2. Policies were optimized with PPO; the stage-2 walking policy and the jumping policy were trained for 8k and 10k iterations, respectively. Training and planning were performed on a desktop computer with an Intel Core i7-11700@2.5GHz CPU and an RTX 4060s GPU. For evaluation, we built test environments in IsaacGym and Gazebo following the scenarios described below.

Real-World Setup: In real-world experiments, we deployed the complete framework on a Unitree Go1 robot. All policy inference, path publishing, and tracking were computed on a single Jetson Nano board. The external sensor was a 16-line RS-Lidar-16 LiDAR mounted on top of the robot. The command publishing frequency was 200 Hz, and the locomotion policy published actions at 50 Hz. The robot used PD control with proportional and derivative gains of $K_p = 20$ and $K_d = 0.5$, respectively.

4.1. Walking Policy Tracking Result Analysis

We evaluate our walking policy against a policy trained only in stage 1, as shown in Fig. 5A and Fig. 5B, in simulated unstructured environments and in a real-world 30° slope-climbing experiment. Our policy maintains a stable base without abrupt attitude changes when encountering slope variations and adapts to diverse terrain, including steps and slippery obstacles in Fig. 5. In contrast, the baseline is prone

Table 2

Domain Randomization and Curriculum Ranges

Name	Walking	Jump
Payload		$[-1, 3]$ kg
Joint Offsets		$[-0.02, 0.02]$ rad
Motor Strength		$[90, 110]\%$
Ground Friction		$[0.04, 5]$
Ground Restitution		$[0, 0.4]$
Centre of Mass Offset		$[-0.1, 0.1]$ m
System delay		$[0, 40]$ ms
PD Gains Factor		$[90, 110]$
Obstacle Height	/	$[0, 0.2]$ m
Obstacle Distance	/	$[0.25, 0.6]$ m
Slope angle	$[0, 30^\circ]$	/
	frequency: $[10, 12]$	frequency: $[2, 3]$
	fractalOctaves: 3	fractalOctaves: 3
Perlin parameters	fractalGain: $[0.25, 0.35]$	fractalGain: $[0.25, 0.35]$
	zScale: $[0.1, 0.25]$	zScale: $[0.1, 0.15]$

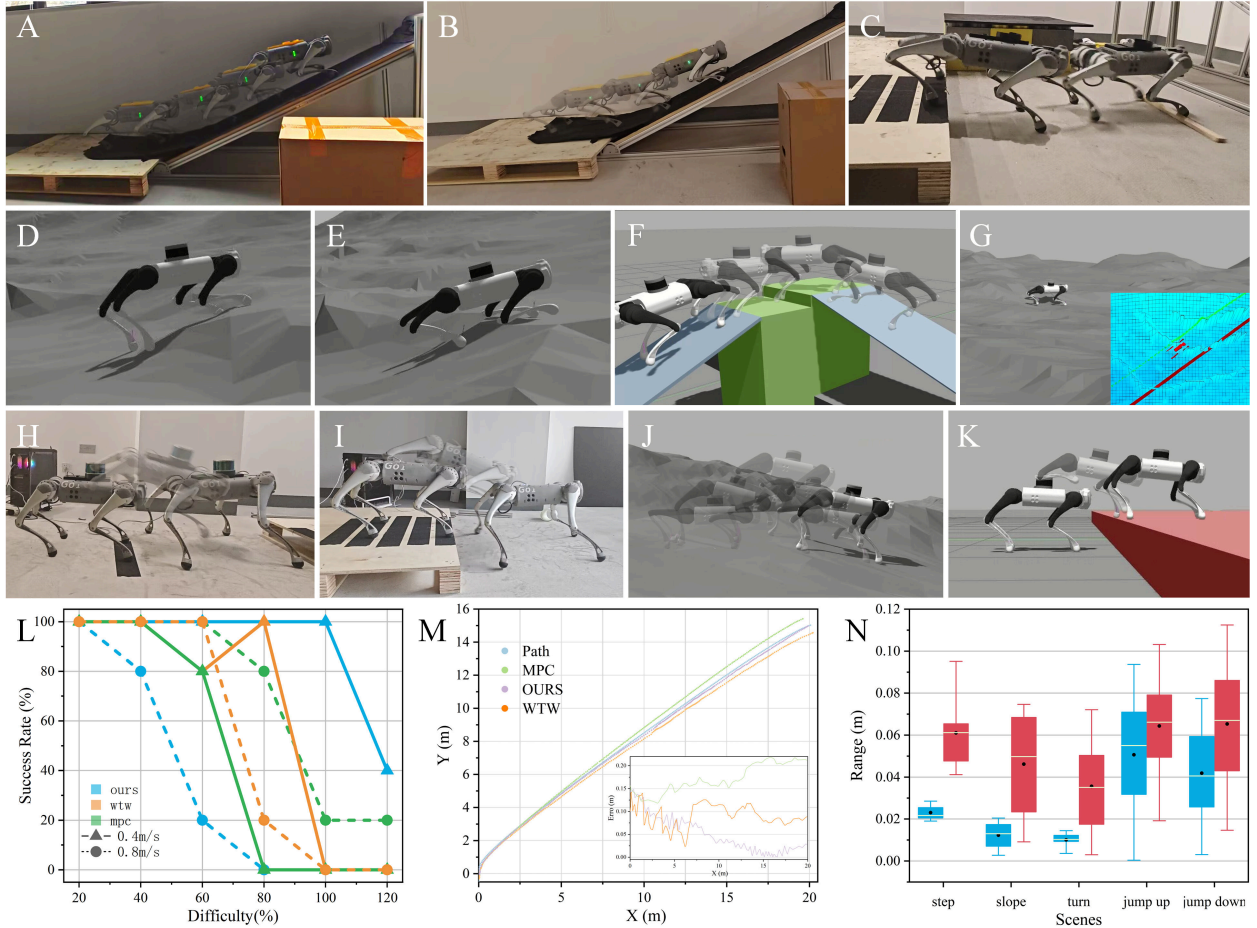


Figure 5: Modal Optimization and Tracking Performance Evaluation: (A)(B) Deploy our walking policy and stage-1 walking policy for comparison on a 30° slope; (C) Navigating a 15cm step and a slippery obstacle; (D)(E) Deploy our walking policy and stage-1 walking policy for comparison in unstructured environments; (F) Tracking success rate experiments on slopes, steps, and low-friction turning scenarios; (G) Long-distance tracking experiment in unstructured environments; (H) Real-world 90cm jump with optimized orientation tracking; (I) Real-world scenario of jumping down a step; (J) Stable jumping on unstructured terrain in simulation; (K) Jumping onto a 20cm obstacle in simulation; (L) Tracking success rate comparison experiment results; (M) Comparative results of long-distance within-corridor tracking rate in unstructured environments; (N) Sim-to-Real tracking errors for walking and jump modes across different scenarios (blue: simulation, red: real-world). The black circle denotes the sample mean, the white horizontal line indicates the median, and the whiskers represent the maximum and minimum observed values, respectively.

to joint-terrain collisions in unstructured environments in Fig. 5D and Fig. 5E.

To evaluate tracking robustness across terrain difficulty, we construct a standardized test platform in Gazebo (Fig. 5(F)) composed of two 2×0.5 m ramps, a 0.5×0.5 m variable-friction flat patch, and a step. We define a difficulty coefficient in $[0, 100]\%$ by varying ramp slope (10° – 30°), step height (3–15 cm), and surface smoothness (0–100%). In Gazebo, surface friction depends on parameters such as μ , μ_2 , slip1 , k_p , and k_d ; we map 0% smoothness to concrete-like friction and 100% to ceramic-tile-like friction. The robot must climb the ramp, turn on the low-friction patch, traverse the step, and descend the ramp without leaving the narrow platform. We compare our stage-2 policy with the stage-1 baseline at two commanded speed settings and run five trials per setting. As shown in Fig. 5(L), our policy achieves a higher success rate and remains reliable even when difficulty exceeds the curriculum range used in training. At higher speeds, most failures are caused by accumulated path deviation rather than inability to overcome the terrain elements.

To evaluate long-distance tracking in unstructured terrain, we construct a $30 \text{ m} \times 30 \text{ m}$ randomized terrain environment in Gazebo and plan a 25 m route (Fig. 5(G)). We compare against two baselines: Walk-These-Ways (WTW) with RMA, and an MPC whole-body controller (WBC). For learning-based methods, we report the mean over 10 trials. The MPC baseline frequently fails to complete the full route; following common practice, we report its best successful trial and explicitly mark incomplete trials as failures in the success-rate statistics.

We define the *within-corridor tracking rate* as the percentage of trajectory samples whose lateral deviation from the planned path stays within a fixed corridor of 0.2 m. Under this metric, our method achieves an average within-corridor tracking rate of 98.61% without failures due to excessive deviation. We attribute residual errors mainly to LiDAR-odometry drift induced by transient body instability; compared with the baselines, our stage-2 policy reduces abrupt roll/pitch changes, which mitigates yaw drift in odometry and improves long-horizon tracking.

4.2. Jumping Policy Result Analysis

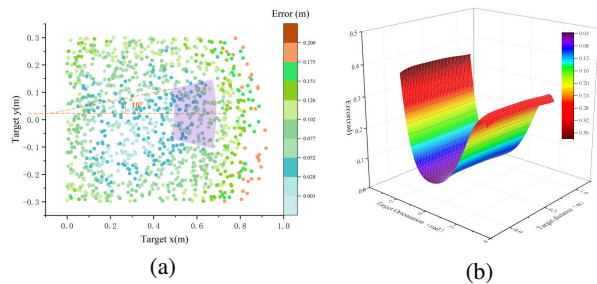


Figure 6: Tracking performance: (a) Jump distance tracking error color scale scatter plot; (b) Jump orientation tracking error 3D Gaussian fitting plot.

Table 3

Mean jump tracking error (m)

Terrain	Ours	Ours W/O planner	CRL	Trajectory
Flat	0.053	0.068	0.062	0.078
Uneven	0.047	0.063	0.077	0.082
10cm Step	0.061	0.072	0.102	0.117

To verify the landing-target tracking capability of our trained policy in unstructured terrain, we conduct 2000 sets of jumping experiments in a gradient noise terrain environment. Fig. 6(a) shows a color-coded scatter plot of the jump tracking error. We set the jump range to Y-axis direction $[-0.3\text{m}, 0.3\text{m}]$ and X-axis direction $[0, 1.0\text{m}]$. Among the 2000 experiments, the robot failed in 12 cases. The robot demonstrates low tracking error for target jumps between 0.1m and 0.7m. Since we require the robot to jump onto obstacles, it must jump a distance exceeding its body length, i.e., the jump distance must be over 0.5m. Analysis of these jump experimental results shows that the optimal tracking range for the robot is within a yaw angle of 10° and a jump distance between 0.5m and 0.7m. Therefore, we apply these limits to the planner for jumps. Fig. 6(b) shows the tracking of heading commands during the 2000 experiments. We observe that heading (yaw) command tracking is largely independent of jump distance and remains reliable for yaw commands within 90° . Therefore, we do not introduce additional distance-dependent heading correction; instead, heading-related constraints are handled at the planner level by enforcing feasibility bounds on the takeoff/landing direction.

To highlight the landing-target tracking performance of our jumping framework, we compare our method with our jumping policy without planner-side pre-jump optimization, Curriculum-Based Reinforcement Learning (CRL) [23], and an MPC-based jumping method with trajectory tracking, in scenarios of flat ground, uneven terrain, and jumping onto a 10 cm obstacle. The results are shown in Table 3. It can be seen that while our strategy's tracking performance on flat ground is comparable to other methods, it maintains relatively stable tracking performance on unstructured terrain and ensures no falling upon landing (Fig. 5(J)), which is difficult for other methods to achieve. Regarding jumping onto obstacles, the robot can leap onto a 20 cm step (Fig. 5(K)).

4.3. Planner Optimize Result Analysis

Our algorithm outperforms the KD-RRT baseline across all five metrics shown in Fig. 7. Compared to KD-RRT, our optimized approach yields a shorter average path length and results in smaller elevation changes along the terrain, which reduces unnecessary vertical COM adjustment and improves travel stability for the quadruped robot.

Specifically, the average path length is reduced by 5.8%. The robot's COM variation is reduced by 31.4%, decreasing the need for the robot to adjust its COM height during locomotion. In terms of terrain metrics, the terrain elevation and terrain slope of the optimized path are reduced by 13.4% and

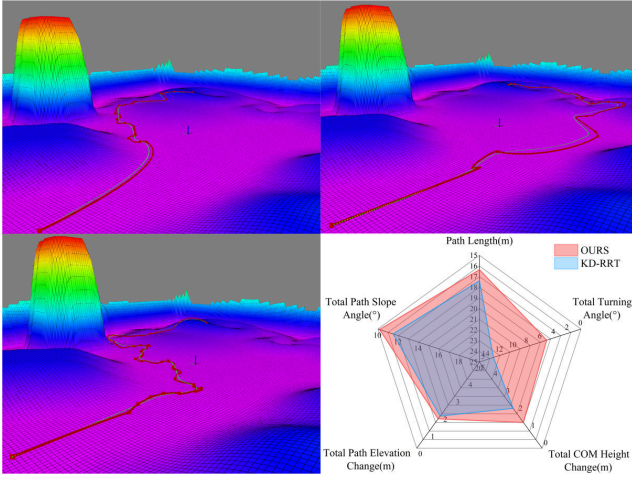


Figure 7: Path optimization comparison. For planning between fixed start and goal on uneven terrain, the red continuous path is the KD-RRT result, while the green discrete-point path is the optimized result. We compare the two in terms of path length, elevation change, turning angle, slope variation, and COM-height variation.

11.4% respectively compared to KD-RRT, enabling traversal of less rugged terrain. Regarding directional changes, our approach reduces turning variation by 62.7% compared to KD-RRT, mitigating frequent turns and large turning angles and resulting in a smoother overall path.

We also report computational cost. Over 20 trials in the same test scene, the mean planning time of our method is 2.18 ± 2.05 s, which is comparable to KD-RRT at 2.30 ± 2.20 s. Although the runtime difference is small, our method yields a clear improvement in path quality.

Based on the operating range indicated by the jump-tracking experiments, we optimize the jump takeoff point and the path direction before and after each jump. Fig. 8A shows the KD-RRT baseline output. The planned takeoff point can be too close to the obstacle boundary, making collisions likely when jumping onto the obstacle. Therefore, we keep the takeoff point 0.2–0.3 m away from the obstacle boundary. Subsequently, based on the reliable tracking range, we restrict the target landing-direction deviation to within 10° . The angle-optimization result is shown in Fig. 8B, and the final optimization result is shown in Fig. 8C.

For the dual-condition trigger, we conduct experiments on jump starting point localization error. In different scenarios, the robot is commanded to switch to the fix-stand mode at the jump starting point. The error between its stabilized position and the required jump starting point is calculated. We compare our dual-condition trigger with single gate detection and single distance detection in terms of positional error and orientation error. The results are shown in Table 4. Our trigger achieves low jump starting point localization error, while single-condition methods are prone to errors such as missing or premature triggering.

Table 4

Jump starting point localization error (e_x, e_y in m; e_r in rad)

Variable	Double	Gate	Distance
e_x	0.052 ± 0.036	0.232 ± 0.167	0.26 ± 0.069
e_y	0.018 ± 0.006	0.072 ± 0.058	0.14 ± 0.051
e_r	0.055 ± 0.048	0.423 ± 0.375	0.733 ± 0.628

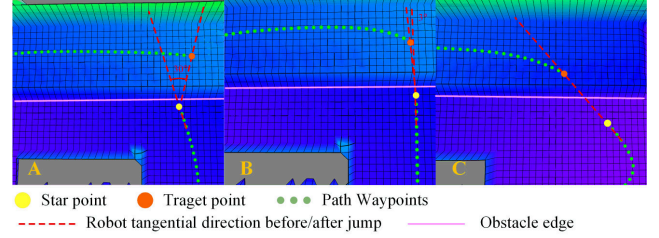


Figure 8: Jump planning optimization results: (A) KD-RRT baseline output; (B) Optimized trajectory with hard orientation constraints; (C) Further refined path incorporating soft orientation constraints and takeoff point optimization.

4.4. Comparison and Ablation Experiments

We evaluate the proposed framework against the state-of-the-art baseline Extreme Parkour [25]. To better quantify performance, we first conduct experiments in the composite scenarios containing jumping (Fig. 9A). In addition, we rebuild in Gazebo the standardized test platform introduced earlier at 100% difficulty. Since Extreme Parkour does not perform path planning—it navigates toward a preset goal using depth-image input—for fairness we provide it with the same waypoints as those produced by our planner. Fig. 9(D) and Fig. 9(E) show the elevation maps of the two composite scenarios together with the planned waypoints. Beyond the two composite scenarios, we further build a 5 m-long 30° narrow slope, an 18 cm gap, a slippery turning platform, and a 20 cm obstacle jump; each scenario is tested over 20 trials, and we record the success rate and the mean tracking error, defined as the distance between the position where the robot first reaches each waypoint and the waypoint itself. The results are shown in Fig. 9(F). Although the baseline exhibits superior agility, its motion is unstable, and its depth-image-based obstacle perception produces a high misjudgment rate; in particular, on narrow sloped terrain the robot tends to fall or fails to regulate its acceleration, leading to task failure (Fig. 9B and Fig. 9C).

We further benchmark the computational load of our framework and the baseline on the experimental GPU, measuring FLOPs and system latency, as shown in Fig. 9(G). When neither uses visual input, the baseline acts as a base policy that directly exploits privileged height information from the simulator without distillation, whereas our policy relies on historical observations to estimate linear velocity and other states for robustness, making it slightly more complex; consequently, our method incurs somewhat higher computational load and latency. However, once the baseline policy is distilled to accept depth-image input, its network

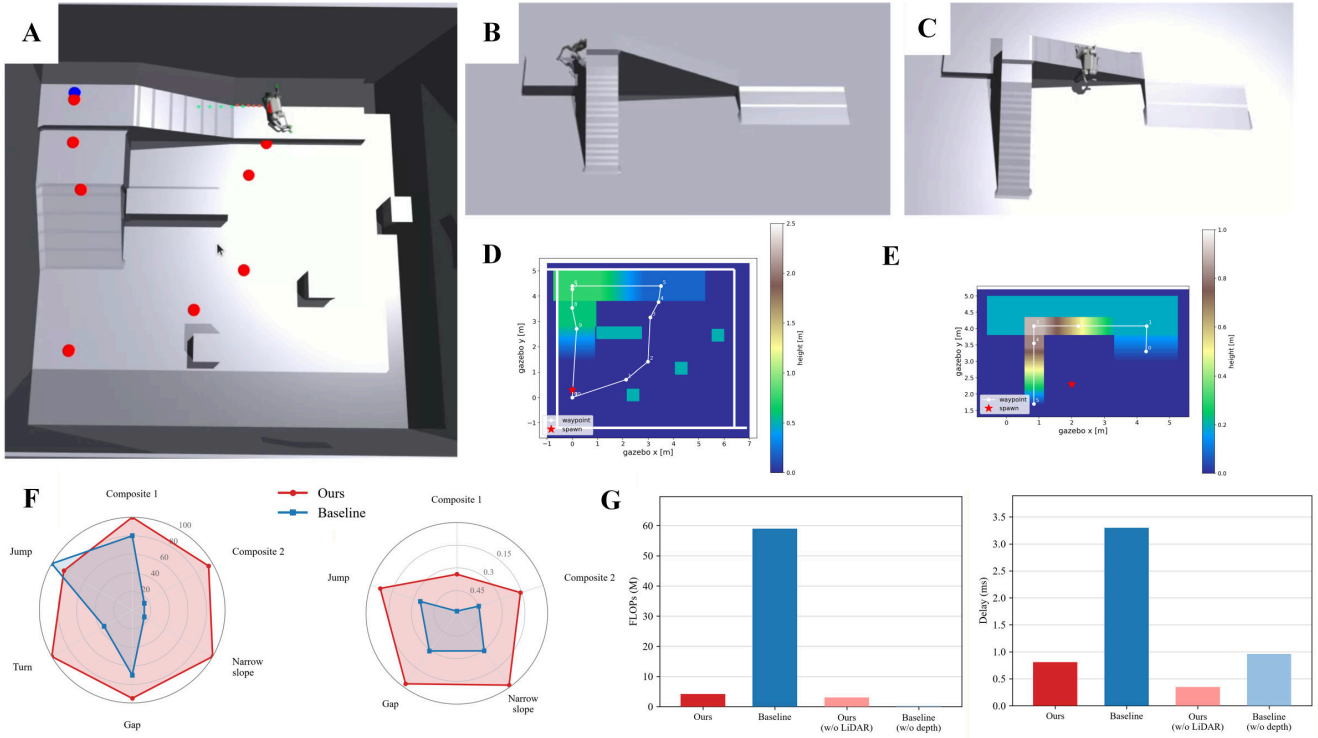


Figure 9: Comparison experiments: (A)(B)(C) Failure cases of the baseline in the two composite test scenarios; (D)(E) Elevation maps of the composite scenarios with the planned paths; (F) Success rate and tracking error across multiple scenarios; (G) Computational complexity (FLOPs) and system latency.

Table 5

System-level ablation in the two composite scenarios (success rate in %, tracking error in m; 20 trials per configuration)

Configuration	Scenario 1		Scenario 2	
	Succ.	Err.	Succ.	Err.
Full framework	100	0.342	100	0.158
w/o path processing	65	0.664	30	0.427
w/o dual-cond. detection	40	0.827	—	—
w/o stab. & perf. rewards	55	0.739	0	—

complexity increases substantially, raising system latency and degrading tracking performance and the success rate. This is a common drawback of current learning-based integrated navigation-and-tracking policies.

In addition to the module-level ablations reported above, we further conduct a system-level ablation in the composite scenarios, disabling one module at each layer of the framework: *w/o path processing* at the planning layer, where the raw KD-RRT output is tracked without discrete optimization, smoothing, or jump-segment refinement; *w/o dual-condition detection* at the tracking layer, where the dual-condition trigger is disabled and mode switching relies on gate detection only; and *w/o stability & performance rewards* at the locomotion layer, where the walking and jumping policies are retrained without the stability- and performance-oriented reward terms. The tracking error and success rate of each configuration are compared in Table 5.

Without path processing, the robot tends to deviate from the landing target during jumps, which leads to large tracking errors in the subsequent path; in a few cases, the take-off position is too close to or too far from the obstacle boundary, resulting in jump failures. In particular, in the narrow terrain of Scenario 2, the raw planner without jump-segment filtering frequently produces jump paths, including hazardous ones that jump directly onto the slope. Without dual-condition detection, the robot misses the takeoff point more frequently, which reduces the task success rate; in Scenario 2, where takeoff points lie on narrow terrain, missed triggers accumulate and no trial completes the full route. Without the stability- and performance-oriented rewards of the walking and jumping policies, the LiDAR point cloud varies frequently during locomotion, and large navigation deviations arise during agile motions, leading to tracking failures; in Scenario 2, none of the trials succeeds.

4.5. Real-World Result Analysis

We construct the scene shown in Fig. 10A. Figures 10B and Fig. 10D show the elevation map constructed from point clouds and the planned path in top-down and front-view perspectives. The disconnected sections in the figure represent jump segments. The endpoint is set at the world-coordinate position (0, 4.4 m) relative to the robot's initial pose. First, the robot climbs onto a wooden platform and walks down a 15 cm step. It then passes through obstacles to reach the takeoff point, jumps onto a 10 cm platform, and finally climbs a 30° slope to reach the endpoint. Fig. 10C

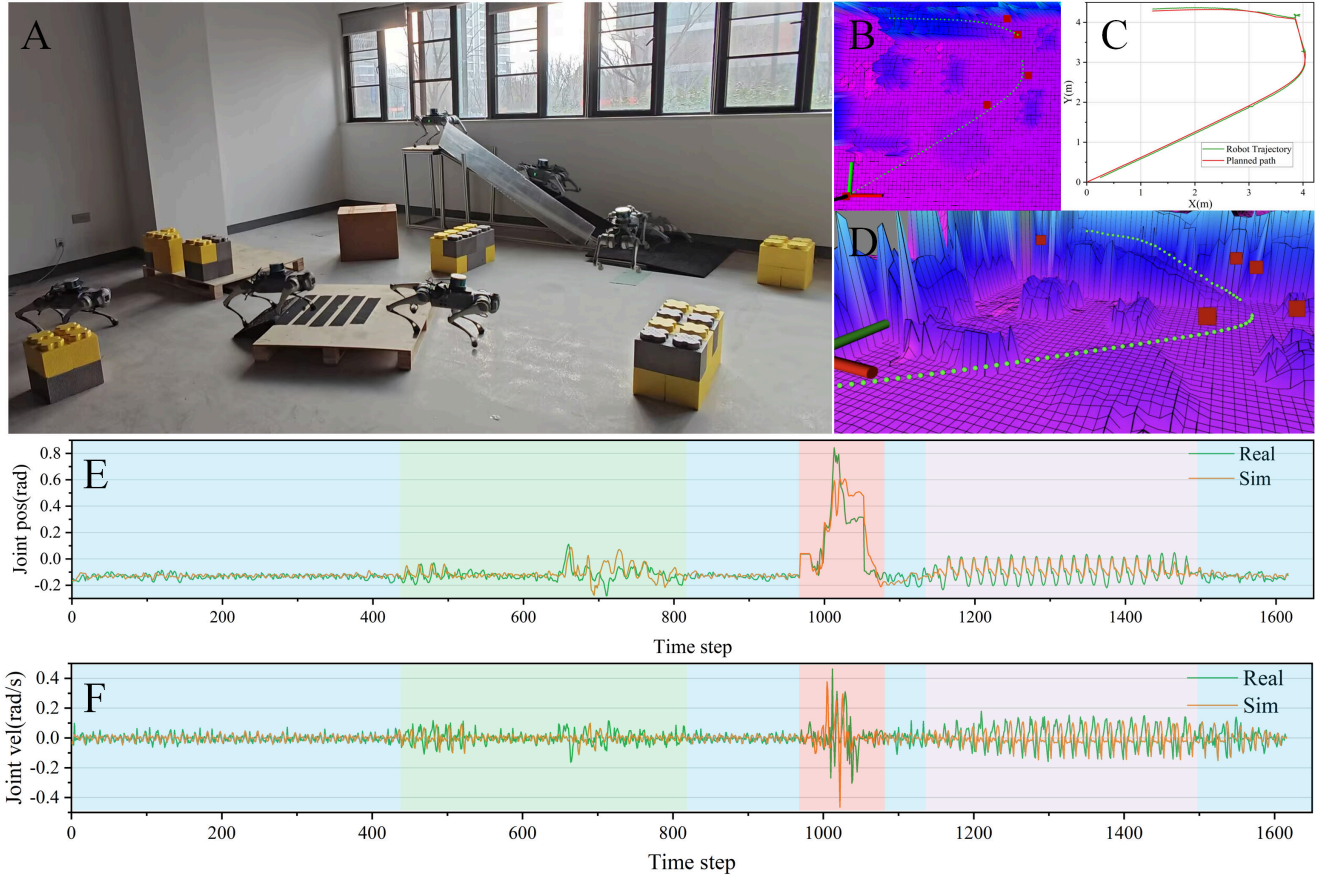


Figure 10: Real-world walking-jumping path planning and tracking: (A) robot navigation and tracking in the test scenario; (B)(D) top-down and front views of the planned path on the elevation map; (C) comparison of the robot's tracked trajectory with the planned path; (E)(F) joint position and velocity of the leg motors during the tracking experiment.

shows the path tracking error, with markers indicating the jump start and end points. These results demonstrate that our method achieves reliable tracking on discrete terrain that requires multiple locomotion modes. Figures 10E and Fig. 10F show the leg joint position and velocity tracking curves during walking, slope ascent/descent, and jumping along this path. By leveraging the MNN-based inference framework, our method reduces the sim-to-real gap and improves deployment stability.

To test the jitter reduction capability of the deployment method, we also conduct a jitter experiment. As shown in Fig. 11, compared to direct policy deployment and inference-framework deployment using MNN, our method demonstrates significantly improved stability in both modalities.

5. Discussion And Conclusion

This work addresses multimodal navigation for walking-jumping quadrupeds in discontinuous, unstructured terrain, where a key challenge is aligning high-level planning with the robot's reliable execution capability under limited sensing and onboard compute. We propose a capability-aware planning and tracking framework that tailors

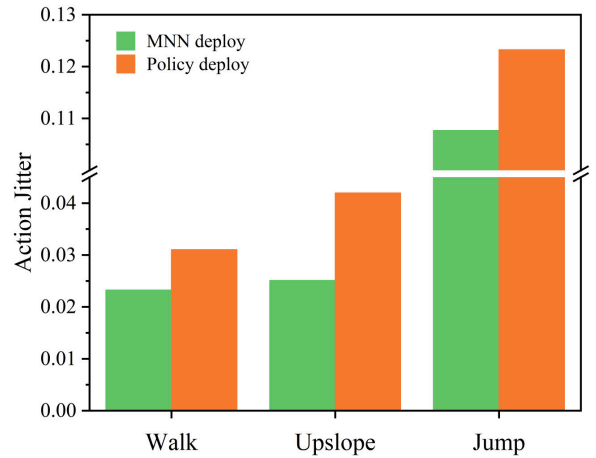


Figure 11: Action jitter. We compute jitter as the mean absolute second-order difference of the commanded target joint positions over 100 control cycles (50 Hz). The unit is $\text{rad}/\text{cycle}^2$ when actions are interpreted as joint position targets.

multimodal path generation to empirically validated jump-tracking feasibility, triggers mode switching with a dual-condition strategy to improve planning-tracking consistency

under LiDAR-odometry uncertainty, and executes the resulting commands with vision-free walking and jumping policies trained via staged curriculum learning and deployed as lightweight state machines. Experiments in simulation and on a real Unitree Go1 demonstrate reliable long-range tracking on complex and discontinuous terrain using only a single LiDAR. A further direction is to improve the jumping capability itself. Our current policy favors landing reliability within the validated range (0.5–0.7 m), whereas recent parkour-style controllers [25, 24] achieve more agile motions, such as leaping across gaps twice the body length, through end-to-end training with rich exteroception. The gap could be closed through reward shaping and curricula that encourage more explosive takeoffs, reference-guided objectives that inject optimized or animal jump trajectories [21, 22], hardware-level enablers such as higher torque limits and foot compliance, and tighter sim-to-real system identification. Combining such agility-oriented training with our capability-aware planning is a promising direction for future work. We also plan to improve the reinforcement learning framework itself. Instead of training walking and jumping as two separate policies coordinated by a state machine, we will investigate their joint training within a single integrated PPO framework. Furthermore, inspired by privileged learning [31] and implicit terrain imagination [32], we will incorporate an autoencoder-based decoder trained with privileged terrain information available in simulation, so that the policy learns to infer the surrounding terrain purely from joint proprioception and LiDAR point-cloud inputs. This is expected to enable more agile locomotion while preserving tracking performance. A current limitation is that the conservative tracking strategy constrains aggressive high-speed behaviors and dynamic obstacle avoidance; improving task completion speed and disturbance robustness remains an important direction for future work.

References

- [1] Norby J, Johnson A M. “Fast global motion planning for dynamic legged robots,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Las Vegas, USA, 2020: 3829–3836.
- [2] Kuffner J J, LaValle S M. “RRT-Connect: An efficient approach to single-query path planning,” in *IEEE International Conference on Robotics and Automation*, San Francisco, USA, 2000: 995–1001.
- [3] Wellhausen L, Hutter M. “Rough terrain navigation for legged robots using reachability planning and template learning,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Prague, Czech Republic, 2021: 6914–6921.
- [4] Xu X, Li P, Zhou J, Deng W. “Path planning of quadrupedal robot based on improved RRT-Connect algorithm,” *Sensors*, 2025, 25(8): 2558.
- [5] Yue L, et al. “Online omnidirectional jump trajectory planning for quadrupedal robots on uneven terrains,” arXiv preprint arXiv:2411.04494, 2024.
- [6] Roth, Pascal, et al. “Viplanner: Visual semantic imperative learning for local navigation,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024.
- [7] Hoeller, David, et al. “Anymal parkour: Learning agile navigation for quadrupedal robots.” *Science Robotics*, 2024, 9(88): eadi7566.
- [8] Kim, Hyeongjun, et al. “High-speed control and navigation for quadrupedal robots on complex and discrete terrain.” *Science Robotics*, 2025, 10(102): eads6192.
- [9] CORMEN T H, LEISERSON C E, RIVEST R L, et al. *Introduction to algorithms*, MIT press, 2022.
- [10] LaValle S. “Rapidly-exploring random trees: a new tool for path planning,” *Research Report*, 1998: No. 9811.
- [11] Kavraki L E, Svestka P, Latombe J C, et al. “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, 1996, 12(4): 566–580.
- [12] Zhang hui, Rong Xue-wen, Li Yi-bin, et al. “Terrain recognition and path planning for quadruped robot,” *Robot*, 2015, 37(5): 546–556.
- [13] Wang, Dewei, et al. “Skill-Nav: enhanced navigation with versatile quadrupedal locomotion via waypoint interface,” in *Vicinagearth*, 2025, Oct 16;2(1):7.
- [14] Zhou, Kangjie, et al. “Adaptive Interactive Navigation of Quadruped Robots using Large Language Models,” arXiv preprint arXiv:2503.22942, 2025.
- [15] Shah D, Osiński B, Ichter B, Levine S. “LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action,” in *Conference on Robot Learning (CoRL)*, 2022.
- [16] Yokoyama N, Ha S, Batra D, Wang J, Bucher B. “VLFM: Vision-language frontier maps for zero-shot semantic navigation,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [17] Cheng A, Ji Y, Yang Z, et al. “NaVILA: Legged robot vision-language-action model for navigation,” in *Robotics: Science and Systems (RSS)*, 2025.
- [18] A. Kumar, Z. Fu, D. Pathak, and J. Malik. “RMA: Rapid motor adaptation for legged robots,” in *Proc. Robot.: Sci. and Syst. (RSS)*, Virtual, July 2021. doi:10.48550/arXiv.2107.04034.
- [19] Narendra, I., Byeongho Yu, and Hyun Myung. “Dreamwaq: Learning robust quadrupedal locomotion with implicit terrain imagination via deep reinforcement learning,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 5078–5084.
- [20] Luo, Shixin, et al. “Pie: Parkour with implicit-explicit learning framework for legged robots.” *IEEE Robotics and Automation Letters*, 2024.
- [21] Zhang, Chi, et al. “Towards jumping skill learning by target-guided policy optimization for quadruped robots.” *Machine Intelligence Research*, 21(6), 2024: 1162–1177.
- [22] Bussola, Riccardo, et al. “Guided reinforcement learning for omnidirectional 3d jumping in quadruped robots,” arXiv preprint arXiv:2507.16481, 2025.
- [23] Atanassov, Vassil, et al. “Curriculum-based reinforcement learning for quadrupedal jump: A reference-free design,” in *IEEE Robotics & Automation Magazine*, 2024.
- [24] Zhuang Z, Fu Z, Wang J, Atanasov N, Schwager M. “Robot parkour learning,” in *Conference on Robot Learning (CoRL)*, 2023.
- [25] Cheng X, Shi K, Agarwal A, Pathak D. “Extreme parkour with legged robots,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [26] Steinbeck, Marcel, and Rainer Koschke. “TinySpline: A small, yet powerful library for interpolating, transforming, and querying NURBS, B-splines, and Bezier curves,” in *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, IEEE, 2021: 572–576.
- [27] Margolis GB, Agrawal P. “Walk these ways: Tuning robot control for generalization with multiplicity of behavior,” in *Conference on Robot Learning*, 2023, pp. 22–31. PMLR.
- [28] Howard, David, et al. “Assessing evolutionary terrain generation methods for curriculum reinforcement learning,” in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2022, pp. 377–384.
- [29] Lv, Chengfei, et al. “Walle: An End-to-End, General-Purpose, and Large-Scale Production System for Device-Cloud Collaborative Machine Learning” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022 (pp. 249–265).

- [30] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning" in *Conference on Robot Learning*, Jan. 2022, pp. 91–100.
- [31] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain" in *Science Robotics*, vol. 5, no. 47, 2020.
- [32] I. M. A. Nahrendra, B. Yu, and H. Myung, "DreamWaQ: Learning robust quadrupedal locomotion with implicit terrain imagination via deep reinforcement learning" in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 5078–5084.



Zhehan Tao received his B.E. degree in Mechanical Design, Manufacturing, and Automation from Anhui University in 2023. He is currently pursuing a Ph.D. degree in Mechanical Engineering at the University of Science and Technology of China. His research interests include reinforcement learning and robotic path planning and navigation.



Zimeng Li obtained a Bachelor's degree in Robotics Engineering from Anhui University in 2025 and is currently pursuing a Ph.D. in Space and Exploration Engineering at the University of Science and Technology of China. His research interests include robot navigation and planning, as well as the application of large-scale robot models.



Erwin Ang Tien Yew received his M.S. degree in Mechanical Engineering/Mechatronics from the University of Southampton, United Kingdom, in 2022. He is currently pursuing a Ph.D. degree in Biological Engineering Mechanics at the School of Engineering Sciences, University of Science and Technology of China. His research interests include quadruped robot control systems and the design of magnetorheological actuator systems, specifically for robotic actuation.



Shuaishuai Sun received the B.E. degree in mechanical engineering and automation from the China University of Mining and Technology, Beijing, China, in 2011, and the Ph.D. degree in 2016 from the University of Wollongong, N.S.W., Australia. He worked as a postdoctoral researcher at the University of Wollongong, Australia, and as an assistant professor at Tohoku University, Japan. He is currently a professor with the Department of Precision Machinery and Precision Instrumentation, University of Science and Technology of China (USTC), Hefei, China. His research interests include intelligent mechanical and mechatronics systems, system dynamics, smart materials and structures, innovative actuators for locomotive robots, and vibration control. He has published more than 120 journal articles and serves as an associate editor or academic editor for several journals.



Liang Zhang is currently an associate professor at the School of Electrical Engineering and Automation, Anhui University, China. His research interests are mainly around multi-robot or multi-agent systems (MAS/MRS), including distributed estimation, cooperative control, and planning under uncertainty. He received his B.Eng. degree in Automation from Shandong University, China, in 2015 and his Ph.D. degree in the School of Astronautics, Harbin Institute of Technology, China, in 2021. He was also a visiting Ph.D. student in the Autonomous System Lab, ETH Zurich, Switzerland, from 2018 to 2019 under the foundation of the China Scholarship Council.